BITS Pilani — Work Integrated Learning Programmes

Machine Learning Operations (MLOps) — AIMLCZG523

Assignment 01

Heart Disease Risk Prediction

End-to-end MLOps pipeline: experiment tracking, CI/CD, containerisation, Kubernetes deployment and monitoring

---

Submitted by

Tilavat Kirtikumar Vrajlal

BITS ID: 2025AE05843

Code repository: https://github.com/KirtiTilavat/MLOps-Assignment1

Run on Prayog Shala (VM) Video :

https://drive.google.com/file/d/1CJY7G9452Och1zvAdQ-zhqf1kTvR7PpQ/view?usp=sharing

Run on local Machine Video :

https://drive.google.com/drive/folders/1lAx2H59lYzv8wzuw88FiyjeO3agFfBUg?usp=sharing

October 2026

1. Introduction

The goal of this assignment is to build a classifier that predicts the risk of heart disease from routine clinical measurements, and to take it all the way to a monitored, containerised API using MLOps practices. The work covers the full lifecycle: reproducible data acquisition, exploratory analysis, model development with cross-validated tuning, experiment tracking in MLflow, automated testing and CI/CD on GitHub Actions, Docker packaging, deployment to Kubernetes, and request logging with Prometheus/Grafana monitoring.

Every step is scripted and can be rerun from a clean checkout with the pinned requirements file. Table 1 maps each assignment task to where it is implemented in the repository.

Table 1: *Assignment tasks and where they are implemented*

#	Task	Implementation in the repository
1	Data acquisition & EDA	`scripts/download_data.py`, `src/data_processing.py`, `notebooks/01_eda.ipynb`
2	Feature engineering & models	`src/features.py`, `src/train.py`, `notebooks/02_training.ipynb`
3	Experiment tracking	MLflow (SQLite backend `mlflow.db`, artifacts in `mlartifacts/`), model registry
4	Packaging & reproducibility	`models/model.joblib` (full sklearn Pipeline), `metadata.json`, `requirements*.txt`
5	CI/CD & testing	`tests/` (38 pytest tests), `.github/workflows/ci.yml`
6	Containerisation	`app/` (FastAPI), multi-stage `Dockerfile`
7	Production deployment	`k8s/` namespace, Deployment, LoadBalancer Service (Docker Desktop)
8	Monitoring & logging	`app/monitoring.py`, `monitoring/` (Prometheus, Grafana, alert rules)
9	Documentation	`README.md`, this report, `screenshots/`

2. System Architecture

Figure 1 shows the four layers of the solution. Locally, scripts download and clean the data and `train.py` tunes two models, logging everything to MLflow and saving the selected pipeline. Every push to `main` triggers the CI pipeline, which lints, tests, trains and builds and smoke-tests the Docker image. The same image runs on Kubernetes behind a LoadBalancer Service. The API logs every request as JSON and exposes Prometheus metrics, which are visualised in Grafana and checked against alert rules.

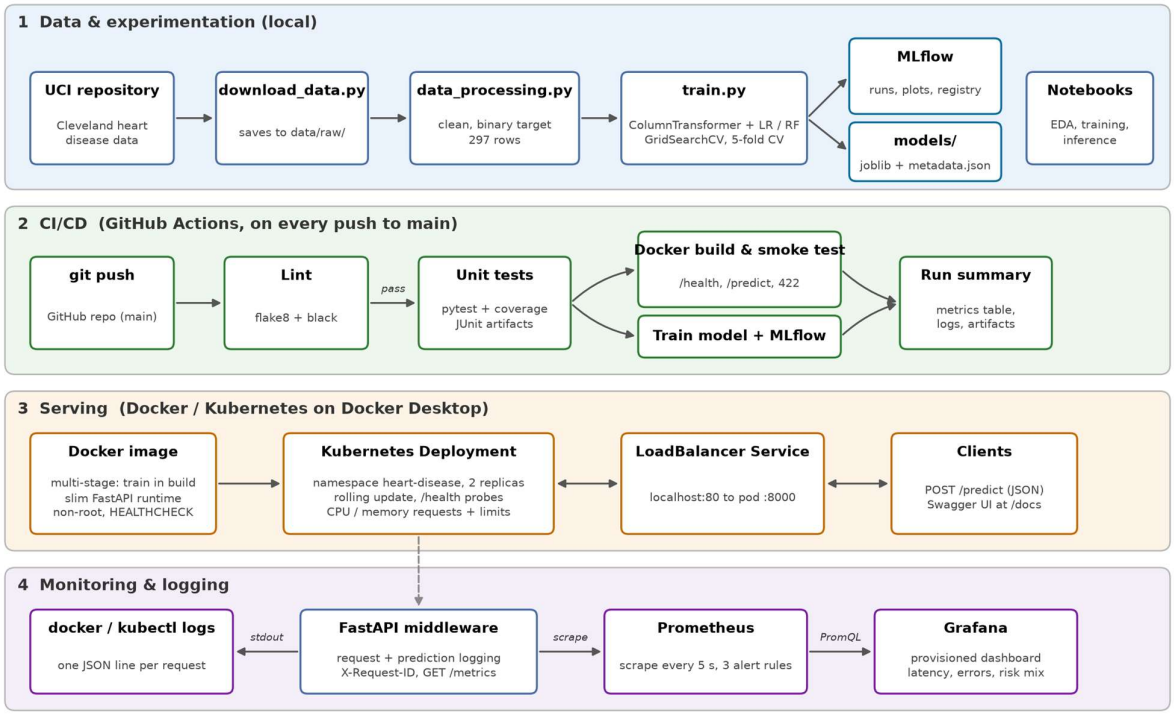


Figure 1: *End-to-end architecture: data and experimentation, CI/CD, serving and monitoring*

3. Setup and Installation

Prerequisites: Python 3.13, Git, and Docker Desktop with Kubernetes enabled (for the container and deployment steps). Everything below is run from the repository root on Windows PowerShell. The same commands work in a Linux shell with `source .venv/bin/activate`.

```
git clone https://github.com/KirtiTilavat/MLOps-Assignment1.git
cd MLOps-Assignment1
python -m venv .venv; .venv\Scripts\Activate.ps1
pip install -r requirements.txt

python scripts/download_data.py          # raw UCI data  -> data/raw/
python -m src.data_processing            # cleaned CSV   -> data/processed/heart_clean.csv
python -m src.train                      # tune, evaluate, log to MLflow, save models/
python -m src.predict                    # score a sample patient with the saved model
pytest -v                                # 38 unit tests
mlflow ui --backend-store-uri sqlite:///mlflow.db   # http://127.0.0.1:5000
```

Running the API, the container, the Kubernetes deployment and the monitoring stack is described in Sections 9–11. Access instructions for local testing are summarised in Section 12.

4. Data Acquisition and Exploratory Data Analysis

4.1 Dataset and cleaning

The data is the **Cleveland subset of the UCI Heart Disease dataset** (`processed.cleveland.data`). It has 303 patients, 13 clinical attributes (age, sex, chest-pain type, resting blood pressure, cholesterol, fasting blood sugar, resting ECG, maximum heart rate, exercise-induced angina, ST depression, ST slope, number of major vessels and thallium test result) and a diagnosis `num` from 0 to 4. `scripts/download_data.py` downloads the file directly from the UCI repository, so the raw data does not need to be committed.

`src/data_processing.py` applies the cleaning steps:

- assigns the 14 documented column names (the raw file has no header) and reads `?` as missing;
- drops the 6 rows with missing values (4 in `ca`, 2 in `thal`, about 2% of the data). This leaves **297 patients**;
- converts the multi-class `num` to a binary `target` (0 = no disease, 1 = disease present, i.e. `num` > 0);
- casts categorical codes to integers and continuous measurements to floats.

The classes are close to balanced: 160 patients (53.9%) without disease and 137 (46.1%) with disease. A stratified train/test split is therefore enough and no resampling is needed.

4.2 Key findings

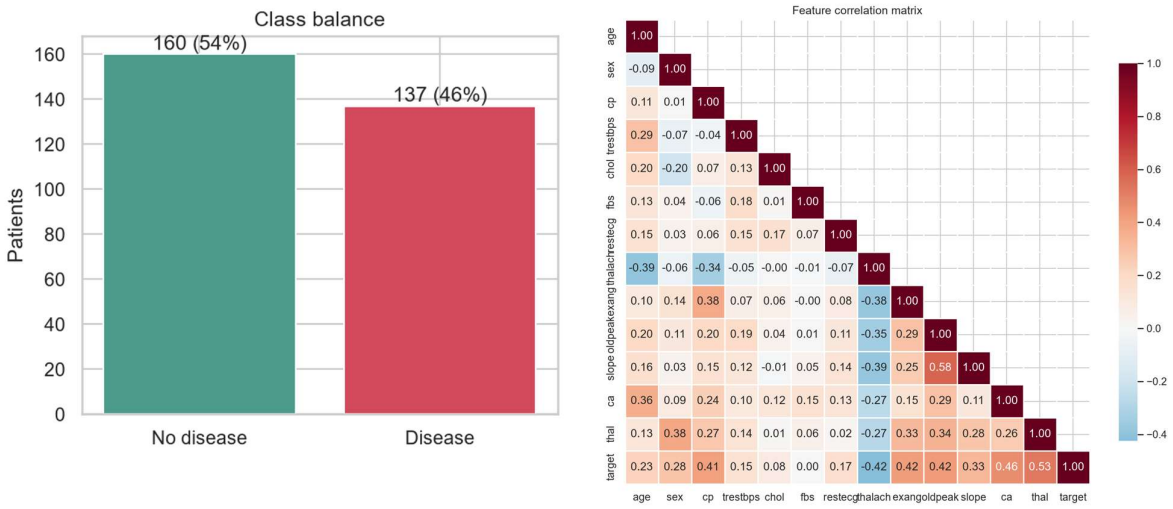


Figure 2: *Class balance (left) and correlation heatmap of the cleaned features (right)*

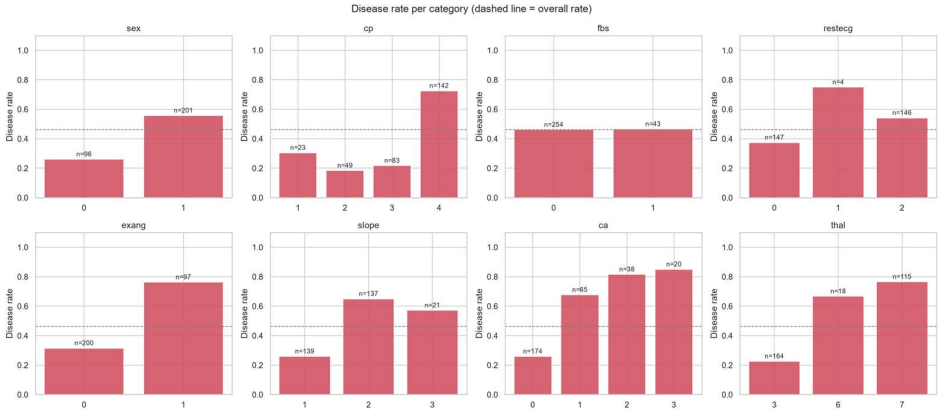


Figure 3: *Disease rate by level of each categorical feature*

- **Continuous features:** patients with disease have a lower maximum heart rate (`thalach`) and higher exercise-induced ST depression (`oldpeak`), and are slightly older. Cholesterol and resting blood pressure separate the classes only weakly.
- **Categorical features:** asymptomatic chest pain (`cp = 4`), exercise-induced angina (`exang = 1`), a reversible thallium defect (`thal = 7`), one or more coloured vessels (`ca > 0`) and male sex all have clearly higher disease rates. These are the strongest signals in the data.
- **Outliers:** a few extreme values exist (e.g. cholesterol above 500 mg/dl), but they are clinically plausible, so they are kept. Standard scaling and the tree model handle them well.

The notebook also contains histograms by class, box plots and an age-versus-heart-rate scatter plot. All figures are saved in `screenshots/eda/`.

5. Feature Engineering and Model Development

5.1 Preprocessing

Preprocessing is a scikit-learn `ColumnTransformer` (`src/features.py`) that is bundled inside the model pipeline. The API therefore receives raw clinical values and applies exactly the same transformations as in training, so there is no training/serving skew.

Table 2: *Feature preprocessing*

Feature group	Columns	Transformation
Continuous	age, trestbps, chol, thalach, oldpeak	StandardScaler
Binary	sex, fbs, exang	Passed through (already 0/1)
Nominal codes	cp, restecg, slope, ca, thal	One-hot encoding (`handle_unknown="ignore"`)

5.2 Models, tuning and selection

The data is split 80/20 (237 training and 60 test patients), stratified on the target with a fixed random seed. Two model families are tuned with `GridSearchCV` using **5-fold stratified cross-validation** on the training set only:

- **Logistic Regression** (liblinear): `C` ∈ {0.01, 0.1, 1, 10}, penalty ∈ {L1, L2}, class_weight ∈ {none, balanced}, giving 16 candidates.
- **Random Forest**: n_estimators ∈ {100, 300}, max_depth ∈ {none, 4, 8}, min_samples_leaf ∈ {1, 3, 5}, class_weight ∈ {none, balanced}, giving 36 candidates.

Every candidate is scored on accuracy, precision, recall, F1 and ROC-AUC. **Recall is the tuning and selection metric**: in a screening setting, missing a patient who has heart disease (a false negative) costs far more than a false alarm that leads to further tests. The final model is chosen on mean CV recall, with ROC-AUC as a tie-breaker. The held-out test set is used only once, for the final report, so it remains an unbiased estimate.

Table 3: *Cross-validated and held-out test performance of the tuned models*

Model (best parameters)	Set	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (C=0.1, L2, balanced)	CV	0.839	0.839	**0.815**	0.821	0.902
	Test	0.867	0.885	0.821	0.852	0.938
Random Forest (300 trees, min_leaf=5, balanced)	CV	0.835	0.843	0.806	0.816	0.902
	Test	0.833	0.846	0.786	0.815	0.941

Logistic Regression was selected. Its CV recall (0.815) is higher than the Random Forest's (0.806), and the two are tied on ROC-AUC (≈0.90). On the test set it reaches 86.7% accuracy, 0.821 recall and 0.938 ROC-AUC. Strong L2 regularisation (C = 0.1) and balanced class weights performed best, which suits a small dataset of 237 training rows. The linear model is also easier to explain to clinicians and is very fast to serve.

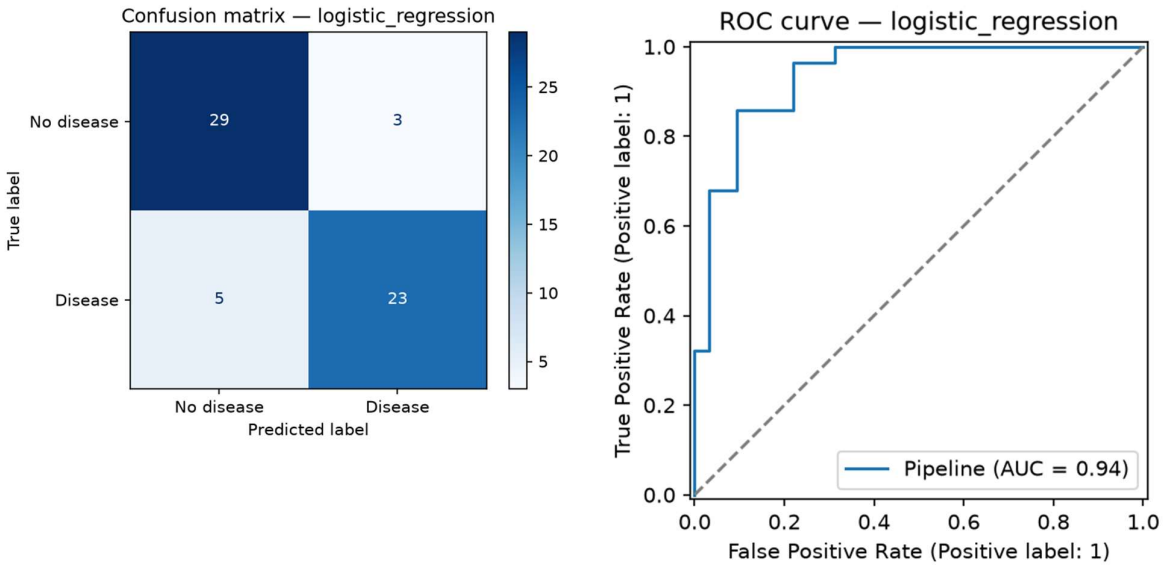


Figure 4: *Selected model on the test set: confusion matrix (left) and ROC curve (right)*

For the API, the predicted probability is also mapped to a readable risk level: **low** (< 0.35), **moderate** (0.35–0.65) and **high** (≥ 0.65).

6. Experiment Tracking with MLflow

All training runs are tracked in MLflow under the experiment `heart-disease-classification`. The tracking backend is a local SQLite database (`mlflow.db`) and artifacts are stored in `mlartifacts/`. The runs are organised as follows:

- **One parent run per model family.** It logs the best hyperparameters, the CV setup (folds, test size, seed, number of candidates), all five CV metrics and all five test metrics. Its artifacts are the confusion matrix and ROC curve plots, the full `cv_results` table as CSV, and the fitted pipeline logged with `mlflow.sklearn` together with an input example and signature.
- **One nested child run per grid candidate** (16 + 36 = 52). Each logs that candidate's parameters, mean CV metrics and recall rank, so the whole search can be compared and filtered in the MLflow UI.
- **Model registry.** The selected run is tagged `selected_model=true`, and its model is registered as `heart-disease-classifier`. Every retraining creates a new version.

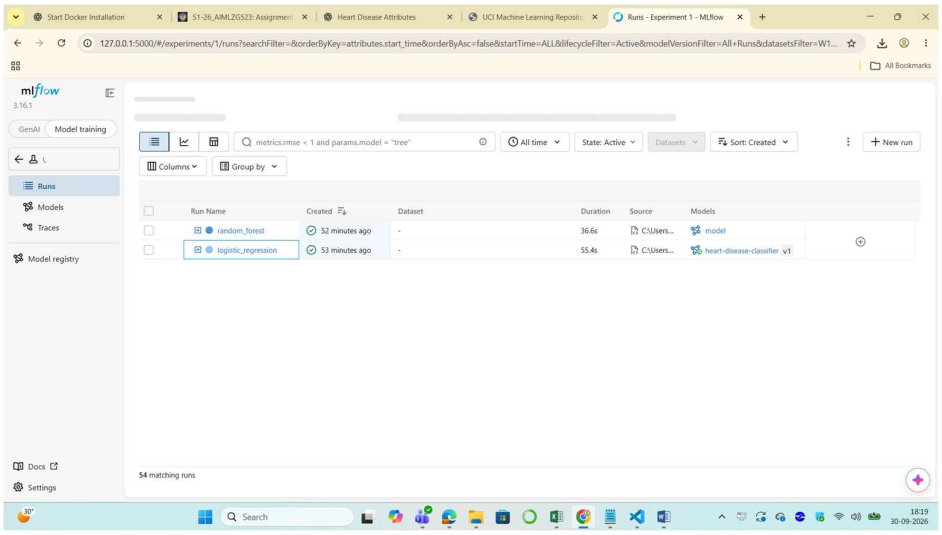


Figure 5: *MLflow runs view for the heart-disease-classification experiment*

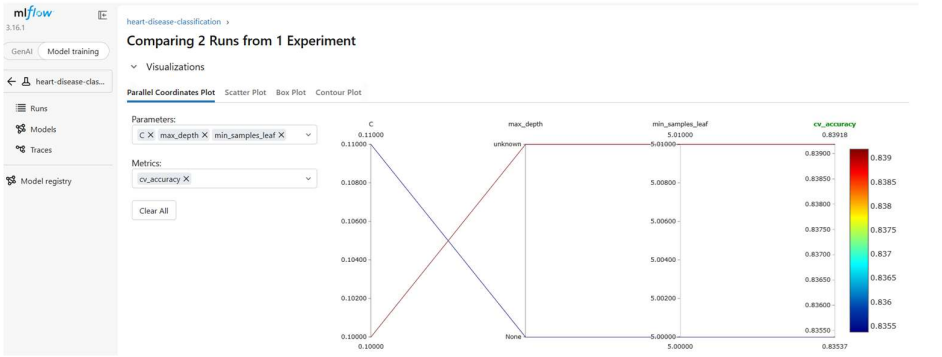


Figure 6: *Comparing the two parent runs: parallel-coordinates plot of parameters and metrics*

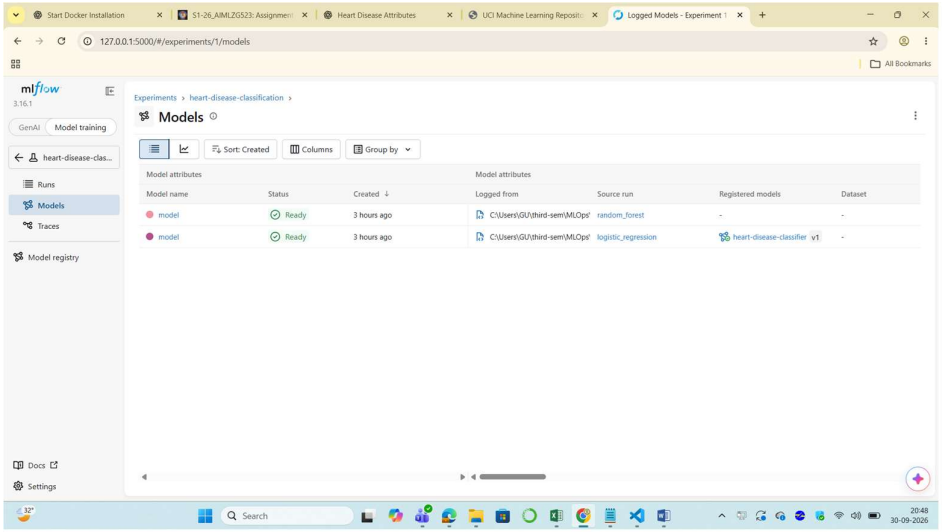


Figure 7: *Logged models and the registered heart-disease-classifier model*

7. Model Packaging and Reproducibility

- **Single artifact.** `models/model.joblib` is the complete scikit-learn `Pipeline` (preprocessor and classifier), so inference needs no separate preprocessing code. The same model is also stored in MLflow format in the registry.
- **Metadata.** `models/metadata.json` records the model name, feature order, best parameters, CV and test metrics for both candidates, and the training timestamp. The API reports this information at `GET /`.
- **Pinned environments.** `requirements.txt` pins every package used for training, testing and notebooks. `requirements-api.txt` is a minimal subset with identical versions for the serving image, so the pickled model loads with exactly the same library versions.
- **Determinism.** A fixed `random_state = 42` is used for the split, the CV folds and both models. The data is downloaded by script rather than copied by hand.
- **Inference.** `src/predict.py` (and `notebooks/03_inference.ipynb`) loads the saved pipeline and scores raw patient records.

8. CI/CD Pipeline and Automated Testing

8.1 Unit tests

The `tests/` folder contains **38 pytest tests** in five modules. They use small synthetic datasets, so they run offline in about 10 seconds:

Table 4: *Unit test coverage by module*

Module	What is tested
test_data.py	`?` parsed as missing, dropping missing rows and duplicates, binary target, column types, input not mutated
test_features.py	all 13 inputs covered once, one-hot output shape, standardisation, unseen categories handled
test_model.py	stratified and reproducible split, valid probabilities, metrics in range, learns signal, grid search runs
test_predict.py	output fields, confidence vs. probability, batch prediction, missing feature raises, risk-level bands
test_api.py	/health, /, /predict, input validation (422), /metrics content, request-ID header and JSON logs

8.2 GitHub Actions workflow

`.github/workflows/ci.yml` runs on every push and pull request to `main`. The jobs are chained, so a failure stops the pipeline early with a clear log:

- **Lint:** `flake8` and `black --check` on all source code.
- **Unit tests:** pytest with coverage. JUnit XML and coverage reports are uploaded as workflow artifacts.
- **Train model** (after tests): downloads the data, cleans it, trains with MLflow tracking, smoke-tests the saved model, writes a metrics table to the job summary and uploads the model and plots as artifacts.
- **Docker build & smoke test** (after tests): builds the image, starts a container, checks `/health`, sends `sample_request.json` to `/predict` and validates the response, confirms that invalid input returns HTTP 422, and prints the container logs.

To show that the pipeline fails on code errors, commit `573fe5c` deliberately changed the low-risk threshold to 0.30. The unit tests caught it and run #2 failed. The fix in run #3 turned the pipeline green again. All later runs have passed.

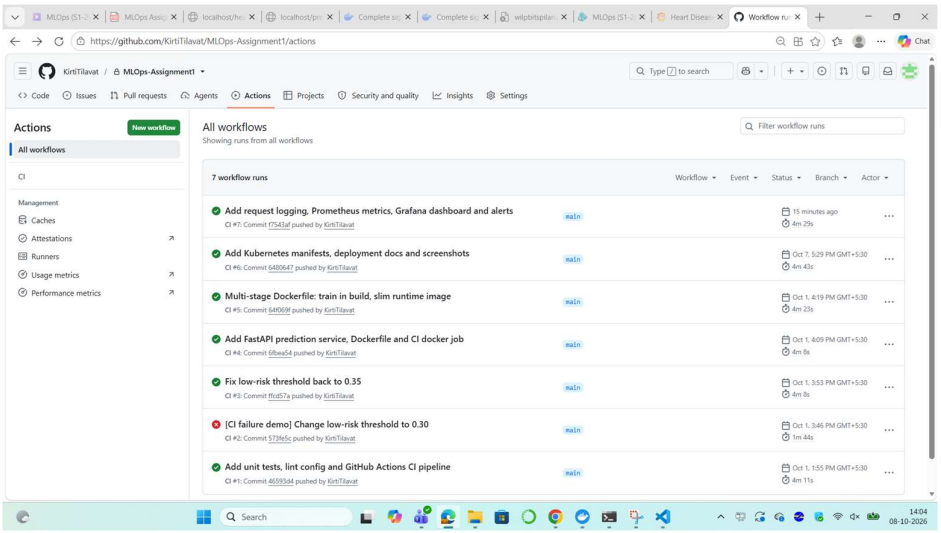


Figure 8: *GitHub Actions history: the deliberately failing run #2 and later successful runs*

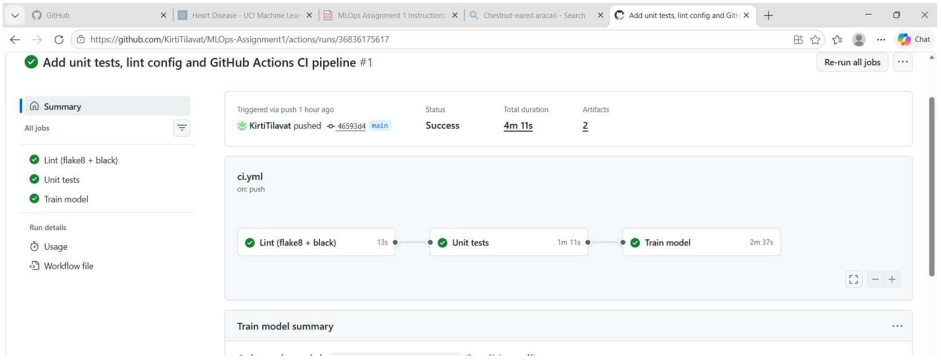


Figure 9: *A successful run: lint, unit tests and training jobs with their artifacts*

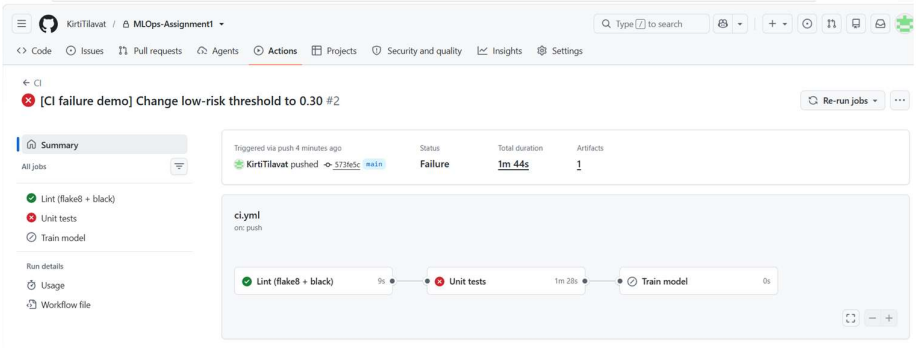


Figure 10: *The failing run #2: the unit test job fails and the later jobs are skipped*

9. Model Containerisation

The serving API is built with **FastAPI** (`app/`). Request bodies are validated with Pydantic: all 13 fields are required, numeric values must be in clinically valid ranges, categorical codes must be valid levels, and extra fields are rejected.

Table 5: *API endpoints*

Endpoint	Description
GET /	Service information: model name, training time, test metrics
GET /health	Liveness/readiness check; returns 503 if the model is not loaded
POST /predict	Accepts patient JSON; returns prediction, label, confidence, probability_disease, risk_level
GET /metrics	Prometheus metrics (Section 11)
GET /docs	Interactive Swagger UI

The `Dockerfile` uses a **multi-stage build**. The **trainer** stage installs the full requirements, downloads the data and trains the model inside the build, so the image never depends on files left over from a developer's machine. The **runtime** stage starts from `python:3.13-slim`, installs only `requirements-api.txt`, and copies in the application code and the trained model. It runs as a non-root user and defines a Docker `HEALTHCHECK` on `/health`. The container was built and tested locally with the sample input:

```
> docker build -t heart-disease-api:1.2 .
> docker run -d -p 8000:8000 --name heart-api heart-disease-api:1.2
> docker images heart-disease-api
REPOSITORY          TAG       SIZE
heart-disease-api   1.2       486MB
> curl -X POST localhost:8000/predict -H "Content-Type: application/json" -d @sample_request.json
{"prediction":1,"label":"disease","confidence":0.9337,"probability_disease":0.9337,
 "risk_level":"high","model_name":"logistic_regression"}
```

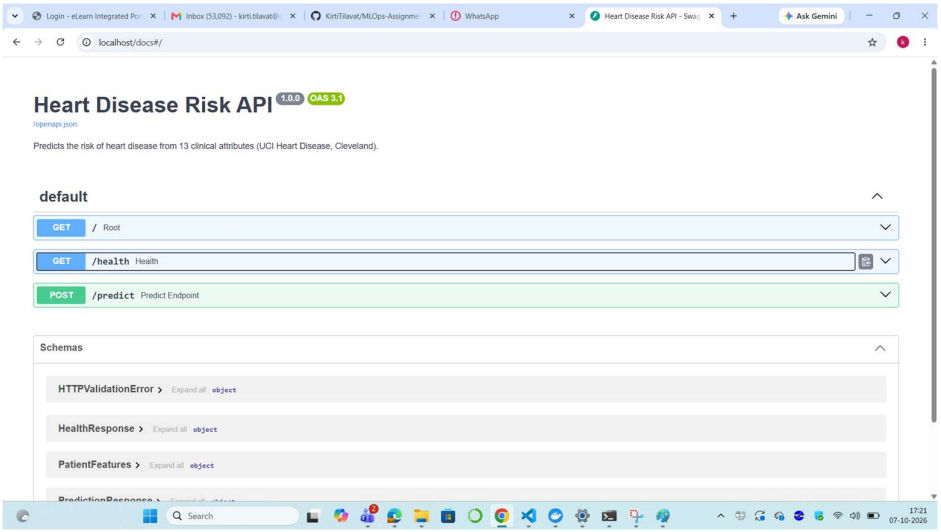


Figure 11: *Swagger UI generated by FastAPI for the prediction service*

10. Production Deployment on Kubernetes

The image is deployed to the local Kubernetes cluster that comes with **Docker Desktop**, using the plain manifests in `k8s/`:

- `namespace.yaml`: a dedicated `heart-disease` namespace.

- `deployment.yaml`: **2 replicas** with a rolling-update strategy (`maxUnavailable: 0`, `maxSurge: 1`), so there is no downtime during updates. It sets CPU/memory requests (100m / 256Mi) and limits (500m / 512Mi), a readiness probe and a liveness probe on `/health`, and runs as non-root. Prometheus scrape annotations are added to the pods.
- `service.yaml`: a **LoadBalancer** Service on port 80 that targets container port 8000. Docker Desktop exposes it at `http://localhost`.

```
kubectl config use-context docker-desktop
docker build -t heart-disease-api:1.2 .
kubectl apply -f k8s/namespace.yaml; kubectl apply -f k8s/
kubectl -n heart-disease rollout status deployment/heart-disease-api

> kubectl -n heart-disease get pods,svc
pod/heart-disease-api-557c96dbd8-cjh9s   1/1   Running   0   42h
pod/heart-disease-api-557c96dbd8-gbl6d   1/1   Running   0   42h
service/heart-disease-api   LoadBalancer   10.107.47.118   localhost   80:30951/TCP
```

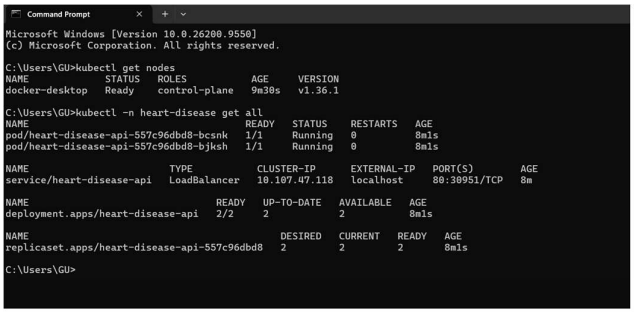


Figure 12: *Kubernetes resources: nodes, two running pods, the LoadBalancer Service and the Deployment*

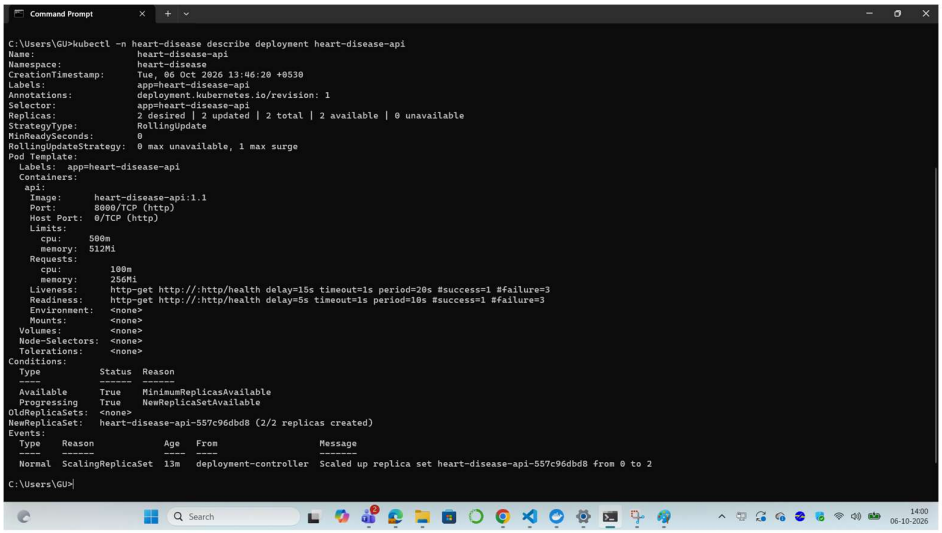


Figure 13: *kubectl describe deployment: replicas, rolling-update strategy, probes and resource limits*

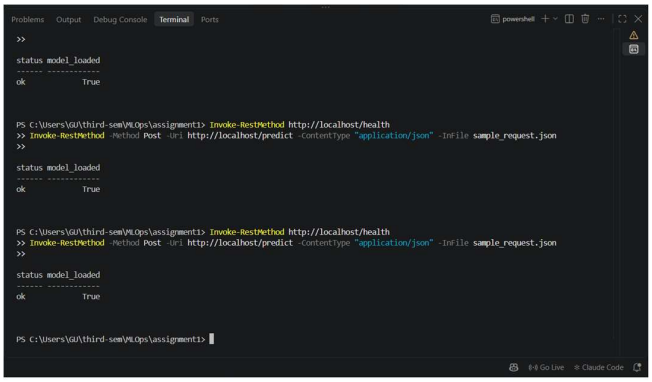


Figure 14: *Endpoint verification through the LoadBalancer: /health and /predict at http://localhost*

11. Monitoring and Logging

11.1 Request logging

A FastAPI middleware (`app/monitoring.py`) writes **one structured JSON line per request** to stdout, with a request ID, method, path, status code, latency and client address. Each `/predict` call also logs a `prediction` event with the input features, predicted class, probability and risk level. Clients may send an `X-Request-ID` header; otherwise one is generated. The ID is returned in the response so that a client call can be traced to its log lines. Probe and scrape traffic (`/health`, `/metrics`) is counted in the metrics but not logged, to keep the logs readable. Because the logs go to stdout, they can be read with `docker logs` or `kubectl logs`, or shipped to any log aggregator.

```
{"ts": "2026-10-08T07:46:30.783792+00:00", "level": "INFO", "logger": "heart_api", "msg":
"prediction",
 "request_id": "26c99bf3cc38", "prediction": 1, "risk_level": "high", "probability_disease": 0.835,
 "features": {"age": 70.0, "sex": 0, "cp": 4, "trestbps": 101.0, "chol": 326.0, ...}}
{"ts": "2026-10-08T07:46:30.784698+00:00", "level": "INFO", "logger": "heart_api", "msg": "request",
 "request_id": "26c99bf3cc38", "method": "POST", "path": "/predict", "status": 200, "duration_ms":
13.91}
```

11.2 Metrics, dashboard and alerts

`GET /metrics` exposes service-level and model-level metrics through `prometheus-client`. Route templates are used as labels, which keeps the number of label values (cardinality) bounded:

Table 6: *Prometheus metrics exposed by the API*

Metric	Type	Purpose
heart_api_requests_total{method,path,status}	Counter	Traffic and error rate per route
heart_api_request_duration_seconds	Histogram	Latency percentiles (p50/p90/p95/p99)
heart_api_requests_in_progress	Gauge	Concurrent requests
heart_api_predictions_total{label,risk_level}	Counter	Prediction mix, a simple drift signal
heart_api_probability_disease	Histogram	Distribution of model scores
heart_api_model_info{model_name,trained_at}	Gauge	Which model version is serving

`monitoring/docker-compose.yml` starts the API together with **Prometheus** (scraping every 5 s) and **Grafana**. The Grafana data source and an 11-panel dashboard are provisioned from files, so they appear without any manual setup. The dashboard shows API status, total predictions, request rate, p95 latency, error rate, mean predicted risk, latency percentiles, predictions per minute by risk level, the overall risk mix and the score distribution. Prometheus also evaluates three alert rules: **ApiDown** (scrape fails for 30 s, critical), **HighErrorRate** (more than 5% 5xx responses for 1 min) and **SlowPredictions** (p95 `/predict` latency above 500 ms for 2 min). `scripts/generate_traffic.py` sends realistic synthetic patients, including about 5% invalid payloads, to produce traffic for the dashboard.

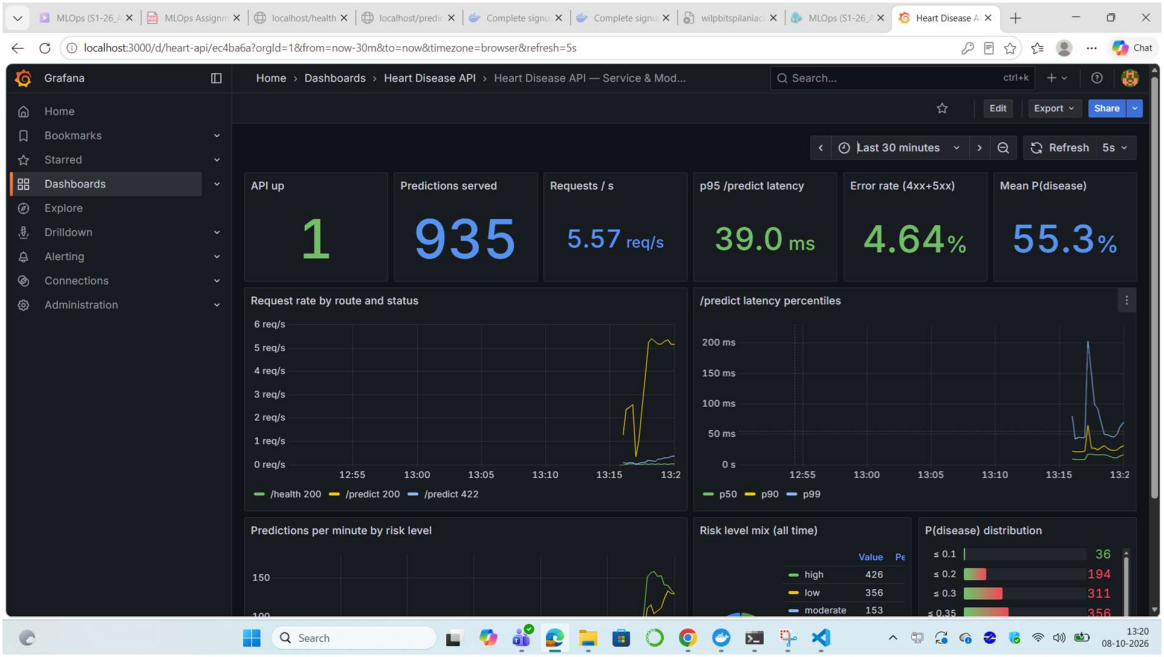


Figure 15: *Grafana dashboard under load: 935 predictions, ~5.6 req/s, p95 latency 39 ms, risk-level mix and score distribution*

To check the alerting, the API container was stopped. The `up` metric dropped to 0, and **ApiDown** became **pending** and then **firing** after 30 seconds (Figure 16). After the container was restarted, the alert cleared automatically.

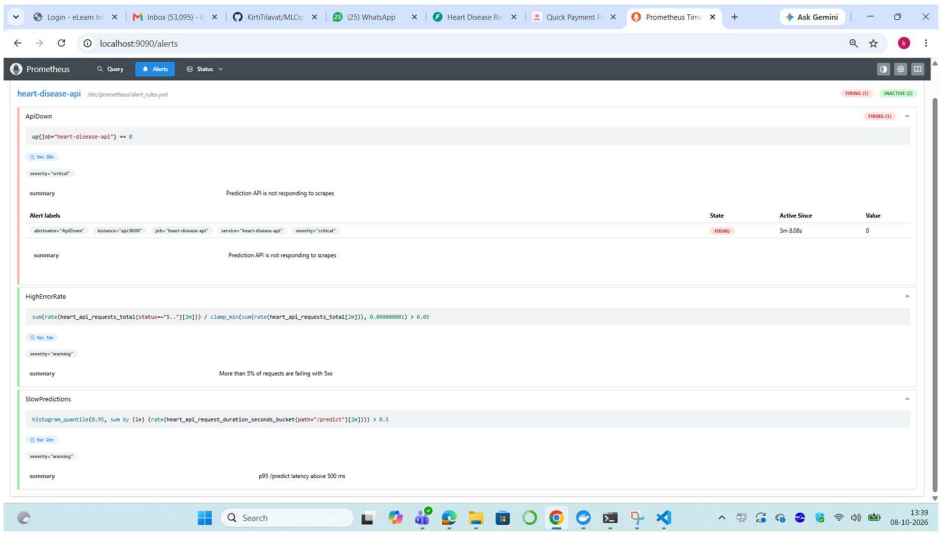


Figure 16: *Prometheus alerts page with ApiDown firing while the API container was stopped*

12. Access Instructions and Conclusion

12.1 Accessing the deployed API (local testing)

- **Kubernetes (Docker Desktop):** `http://localhost`. Use `GET /health`, `POST /predict` with the JSON in `sample_request.json`, or open `/docs` for the Swagger UI.
- **Docker Compose monitoring stack:** API at `http://localhost:8000`, Prometheus at `http://localhost:9090`, Grafana at `http://localhost:3000` (admin / admin).
- **Source code:** https://github.com/KirtiTilavat/MLOps-Assignment1

```
Invoke-RestMethod -Method Post -Uri http://localhost/predict `
  -ContentType "application/json" -InFile sample_request.json
```

12.2 Conclusion and future work

The project delivers a reproducible, tested and monitored heart-disease risk service. A tuned Logistic Regression pipeline reaches 86.7% accuracy and 0.938 ROC-AUC on held-out patients. It is tracked and versioned in MLflow, validated on every commit by a four-stage GitHub Actions pipeline, packaged in a slim non-root Docker image, deployed as a highly available Kubernetes Deployment, and observed through structured logs, Prometheus metrics, Grafana dashboards and alerts.

Natural next steps would be:

- pushing the image to a registry and deploying to a managed cluster (GKE/EKS/AKS) with an Ingress and TLS;
- adding a continuous-deployment stage that rolls out each new image tag automatically;
- running Prometheus inside the cluster (kube-prometheus) and using the existing pod annotations for service discovery;
- adding statistical data-drift checks on the logged features, with scheduled retraining when drift is detected;
- training on the full multi-site UCI data (Hungary, Switzerland, VA) to improve generalisation.